

Theory of Programming Languages

5. Curry-Howard Correspondence

Kihong Heo



Parallel Universe

- Observed by Curry (1934) and extended by Howard (1969)
- Slogan: propositions as types & proofs as programs
- Connection between logic and computation
- Foundation of proof assistants (e.g., Lean, Rocq, etc)

Deja Vu (1)

- Variable & Hypothesis

$$\frac{\Gamma(x) = \tau}{\Gamma \vdash x : \tau}$$

$$\frac{A \in \Gamma}{\Gamma \vdash A} \text{Hyp}$$

- Function & Implication

$$\frac{\Gamma + x : \tau_1 \vdash E : \tau_2}{\Gamma \vdash \lambda x : \tau_1. E : \tau_1 \rightarrow \tau_2}$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} \rightarrow I$$

$$\frac{\Gamma \vdash E_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash E_2 : \tau_1}{\Gamma \vdash E_1 E_2 : \tau_2}$$

$$\frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} \rightarrow E$$

Deja Vu (2)

- Product & Conjunction

$$\frac{\Gamma \vdash E_1 : \tau_1 \quad \Gamma \vdash E_2 : \tau_2}{\Gamma \vdash \langle v_1, v_2 \rangle : \tau_1 \times \tau_2}$$

$$\frac{\Gamma \vdash E : \tau_1 \times \tau_2}{\Gamma \vdash E.1 : \tau_1}$$

$$\frac{\Gamma \vdash E : \tau_1 \times \tau_2}{\Gamma \vdash E.2 : \tau_2}$$

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge I$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge E L$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge E R$$

Deja Vu (3)

- Sum & Disjunction

$$\frac{\Gamma \vdash E : \tau_1}{\Gamma \vdash \text{inl } E : \tau_1 + \tau_2}$$

$$\frac{\Gamma \vdash E : \tau_2}{\Gamma \vdash \text{inr } E : \tau_1 + \tau_2}$$

$$\frac{\Gamma \vdash E : \tau_1 + \tau_2 \quad \Gamma + x : \tau_1 \vdash E_1 : \tau \quad \Gamma + x : \tau_2 \vdash E_2 : \tau}{\Gamma \vdash \text{case } E \text{ of inl } x \Rightarrow E_1 \mid \text{inr } x \Rightarrow E_2 : \tau}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee\text{IL}$$

$$\frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \vee\text{IR}$$

$$\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} \vee\text{E}$$

Deja Vu (4)

- Unit / Void & True / False

$$\frac{}{\Gamma \vdash \langle \rangle : \text{unit}}$$

$$\frac{\Gamma \vdash E : \text{void}}{\Gamma \vdash \text{abort}_{\tau} E : \tau}$$

$$\frac{}{\Gamma \vdash \top} \top\text{I}$$

$$\frac{\Gamma \vdash \perp}{\Gamma \vdash C} \perp\text{E}$$

Proof Term

- Terms in λ -calculus representing proofs in natural deduction
 - Recall: propositions as types & proofs as programs
 - Writing proof = writing programs
- Connection between logic and computation
 - Provable = Inhabited

$$\Gamma \vdash E : A$$

Example

$$A \rightarrow (B \rightarrow A)$$

$$\frac{\frac{\frac{}{A, B \vdash A} \text{Hyp}}{A \vdash B \rightarrow A} \rightarrow\text{I}}{\vdash A \rightarrow (B \rightarrow A)} \rightarrow\text{I}$$

$$\frac{\frac{\frac{}{x:A, y:B \vdash x:A} \text{Hyp}}{x:A \vdash \lambda y:B. x : B \rightarrow A} \rightarrow\text{I}}{\vdash \lambda x:A. \lambda y:B. x : A \rightarrow (B \rightarrow A)} \rightarrow\text{I}$$

Interactive Proof Assistant

- One of the most impactful applications of Curry-Howard correspondence
- Proof checking = type checking
- Proof writing = programming
- Tactic: procedure for constructing proof terms

$$\vdash \lambda h_1. \lambda h_2. (h_1 \ h_2) : (A \rightarrow B) \rightarrow A \rightarrow B$$

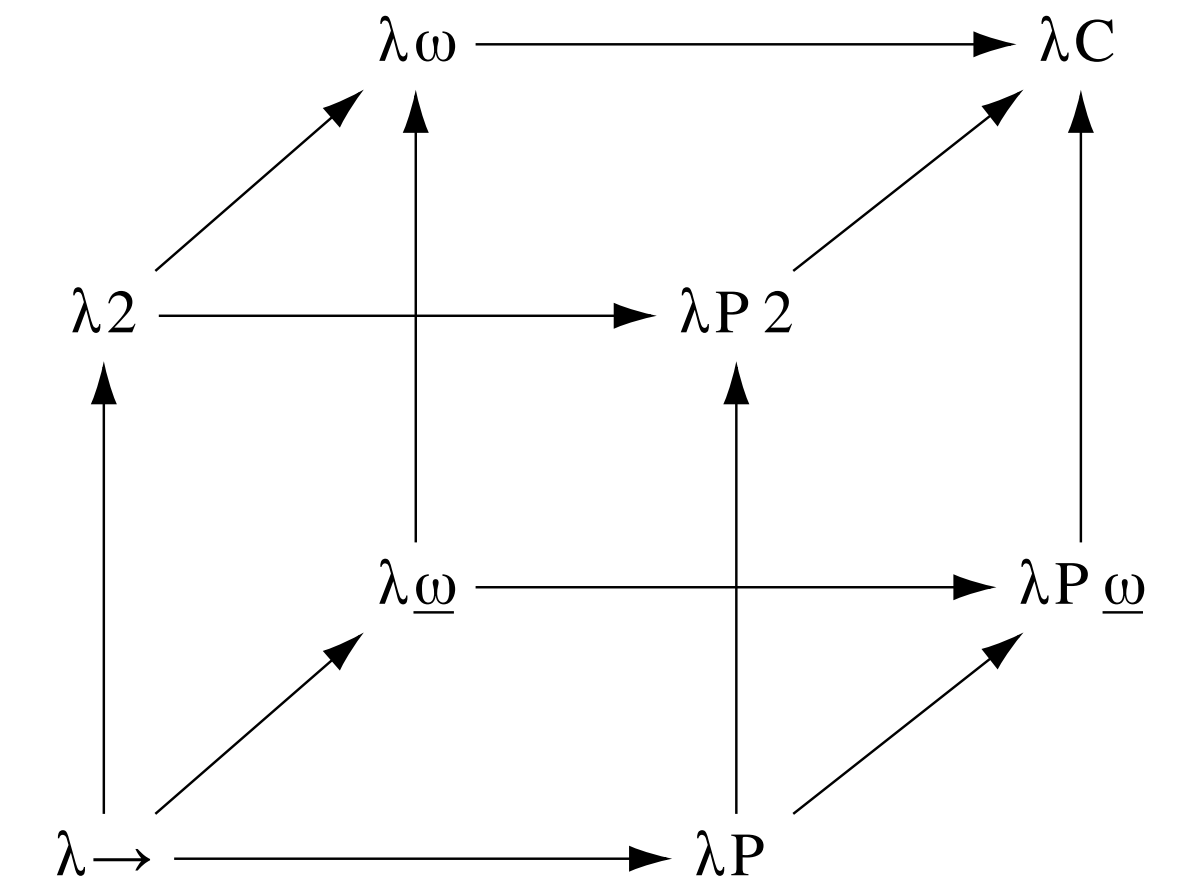
$$\frac{\frac{\frac{h_1 : (A \rightarrow B), h_2 : A \vdash h_1 : A \rightarrow B}{h_1 : (A \rightarrow B), h_2 : A \vdash \$2 \ \$3 : B} \text{intro } h_2}{h_1 : (A \rightarrow B) \vdash \lambda h_2. \$1 : A \rightarrow B} \text{intro } h_1}{\vdash \lambda h_1. \$0 : (A \rightarrow B) \rightarrow A \rightarrow B} \begin{array}{l} \text{exact } h_2 \\ \text{apply } h_1 \end{array}$$

Curry-Howard Correspondence (1)

Programming	Logic
Type	Proposition
Program	Proof
Inhabitation	Provability
Function Type	Implication
Product Type	Conjunction
Sum Type	Disjunction
Type Checking	Proof Checking
Program Synthesis	Proof Synthesis
...	...

Curry-Howard Correspondence (2)

- How about more advanced type systems?
 - Simply typed lambda calculus = propositional logic
- Three dimensions of typed λ -calculus
 - Origin: simple type
 - x-axis: dependent type
 - y-axis: polymorphic type
 - z-axis: type operator



Question

- Correspondence to Classical logic?



Continuation_{마저할일}

- Introduced by Strachey and Wadsworth (1974)
 - “Continuations: A Mathematical Semantics for Handling Full Jumps”, 1974
 - Reynold, “The Discoveries of Continuations”, 1993
- Discovery: What is the mathematical semantics of “goto” in Algol 60?
(also, setjmp/longjmp, callcc)
- Applications: programming with flexible control flow, systematic compilation, etc
- Common concept:
 - “수업 끝나고 집에 가는 길에 **마트에 들러 장**을 봐야지”
“After class, I'll **stop by the grocery store** on my way home.”
 - Call stack

Example (1)

- In $E_1 + E_2$, what remains after evaluating E_1 ?
 - $\lambda x . x + E_2$
- In $E_1 + E_2$, what remains after evaluating E_2 ?
 - $\lambda x . v_1 + x$
- In $E_1 E_2$, what remains after evaluating E_1 ?
 - $\lambda x . x E_2$
- In $E_1 E_2$, what remains after evaluating E_2 ?
 - $\lambda x . f x$

Example (2)

- In $(\text{goto } L) + E$, what remains after `goto`?
 - Code at label L
- In $(\text{raise } L) + E$, what remains after `raise`?
 - Exception handler for exception L

Continuation & Evaluation Context

- In $K[E]$, what remains after evaluating E ?

- $\lambda x . K[x]$

- How to complete the remaining job?

- $(\lambda x . K[x]) \ v$

- Example

- $K = 1 + [] : \tau_E \rightarrow int$

- $K[E] = 1 + E : int$

- $k = \lambda v . 1 + v : \tau_E \rightarrow int$

Continuation Passing Style (CPS)

- Explicitly represent continuation as a term (마저할일 드러내기 의미구조)
- Idea: $\llbracket E \rrbracket$ is a function that takes continuation as a parameter and evaluates E
- Example

$$E \rightarrow n \mid x \mid \lambda x.E \mid E \ E \mid E + E$$

$$\llbracket E \rrbracket \in \mathbb{K} \rightarrow \mathbb{V}$$

$$\llbracket n \rrbracket = \lambda k.k \ n$$

$$\llbracket x \rrbracket = \lambda k.k \ x$$

$$\llbracket \lambda x.E \rrbracket = \lambda k.k \ (\lambda x.\lambda k'.\llbracket E \rrbracket \ k')$$

$$\llbracket E_1 \ E_2 \rrbracket = \lambda k.\llbracket E_1 \rrbracket (\lambda f.\llbracket E_2 \rrbracket (\lambda v.f \ v \ k))$$

$$\llbracket E_1 + E_2 \rrbracket = \lambda k.\llbracket E_1 \rrbracket (\lambda n_1.\llbracket E_2 \rrbracket (\lambda n_2.k(n_1 + n_2)))$$

Exercise

- 1
- $x + 1$
- $\lambda x.x$
- $(\lambda x.x) 1$

Exception in Lambda Calculus?

```
let f x =  
  try raise (Error x)  
  with Error x -> x + 1
```

```
let g x =  
  try x  
  with Error x -> x + 1
```

```
let _ = f 0  
let _ = g 0
```

$$f = \lambda x.(\text{catch } e (\text{throw } e \ x)) + 1$$
$$g = \lambda x.(\text{catch } e \ x) + 1$$
$$f \ 0 : (\text{catch } e (\text{throw } e \ 0)) + 1 \rightarrow 0 + 1 \rightarrow 1$$
$$g \ 0 : (\text{catch } e \ 0) + 1 \rightarrow 0$$

Exception

- Terms

$$\begin{array}{c}
 E \rightarrow \dots \\
 | \text{ catch } x \ E \\
 | \text{ throw } x \ E
 \end{array}$$

- Dynamic semantics

$$\frac{}{K[\text{catch } x \ E] \rightarrow E\{\lambda v.K[v]/x\}}$$

$$\frac{}{K[\text{throw } (\lambda v.E_1) \ E_2] \rightarrow (\lambda v.E_1) \ E_2}$$

$$\begin{array}{lcl}
 \llbracket E \rrbracket & \in & \mathbb{K} \rightarrow \mathbb{V} \\
 \llbracket \text{catch } x \ E \rrbracket & = & \lambda k.(\lambda x.\llbracket E \rrbracket \ id) \ k \\
 \llbracket \text{throw } x \ E \rrbracket & = & \lambda k.\llbracket E \rrbracket \ x
 \end{array}$$

- Static semantics

$$\frac{\Gamma, x:\tau \rightarrow \tau' \vdash E : \tau'}{\Gamma \vdash \text{catch } x \ E : \tau}$$

$$\frac{\Gamma(x) = \tau \rightarrow \tau' \quad \Gamma \vdash E : \tau}{\Gamma \vdash \text{throw } x \ E : \tau''}$$

Exercise

- $(\text{catch } x (\text{throw } x 0)) + 1$
- $(\text{catch } x 0) + 1$

Goto

- Goto simulated using catch

$$\begin{array}{c}
 E \rightarrow \dots \\
 | \text{ catch } x \ E \\
 | \text{ throw } x \ E
 \end{array}$$

- Dynamic semantics

$$\frac{K[\text{catch } x \ E] \rightarrow K[E\{\lambda v. K[v]/x\}]}{K[\text{throw } (\lambda v. E_1) \ E_2] \rightarrow (\lambda v. E_1) \ E_2}$$

$$\begin{aligned}
 \llbracket E \rrbracket &\in \mathbb{K} \rightarrow \mathbb{V} \\
 \llbracket \text{catch } x \ E \rrbracket &= \lambda k. (\lambda x. \llbracket E \rrbracket \ k) \ k \\
 \llbracket \text{throw } x \ E \rrbracket &= \lambda k. \llbracket E \rrbracket \ x
 \end{aligned}$$

- Static semantics

$$\frac{\Gamma, x:\tau \rightarrow \tau' \vdash E : \tau}{\Gamma \vdash \text{catch } x \ E : \tau} \quad \frac{\Gamma(x) = \tau \rightarrow \tau' \quad \Gamma \vdash E : \tau}{\Gamma \vdash \text{throw } x \ E : \tau''}$$

Curry-Howard Correspondence

- Exception

$$\frac{\Gamma, x:\tau \rightarrow \tau' \vdash E : \tau'}{\Gamma \vdash \text{catch } x E : \tau}$$

$$\frac{\Gamma, A \rightarrow \perp \vdash \perp}{\Gamma \vdash A} \text{DNE} = \frac{\Gamma, \neg A \vdash \perp}{\Gamma \vdash A} = \frac{\Gamma \vdash \neg \neg A}{\Gamma \vdash A}$$

$$\frac{\Gamma(x) = \tau \rightarrow \tau' \quad \Gamma \vdash E : \tau}{\Gamma \vdash \text{throw } x E : \tau''}$$

$$\frac{\Gamma \vdash \neg A \quad \Gamma \vdash A}{\Gamma \vdash C} \neg E$$

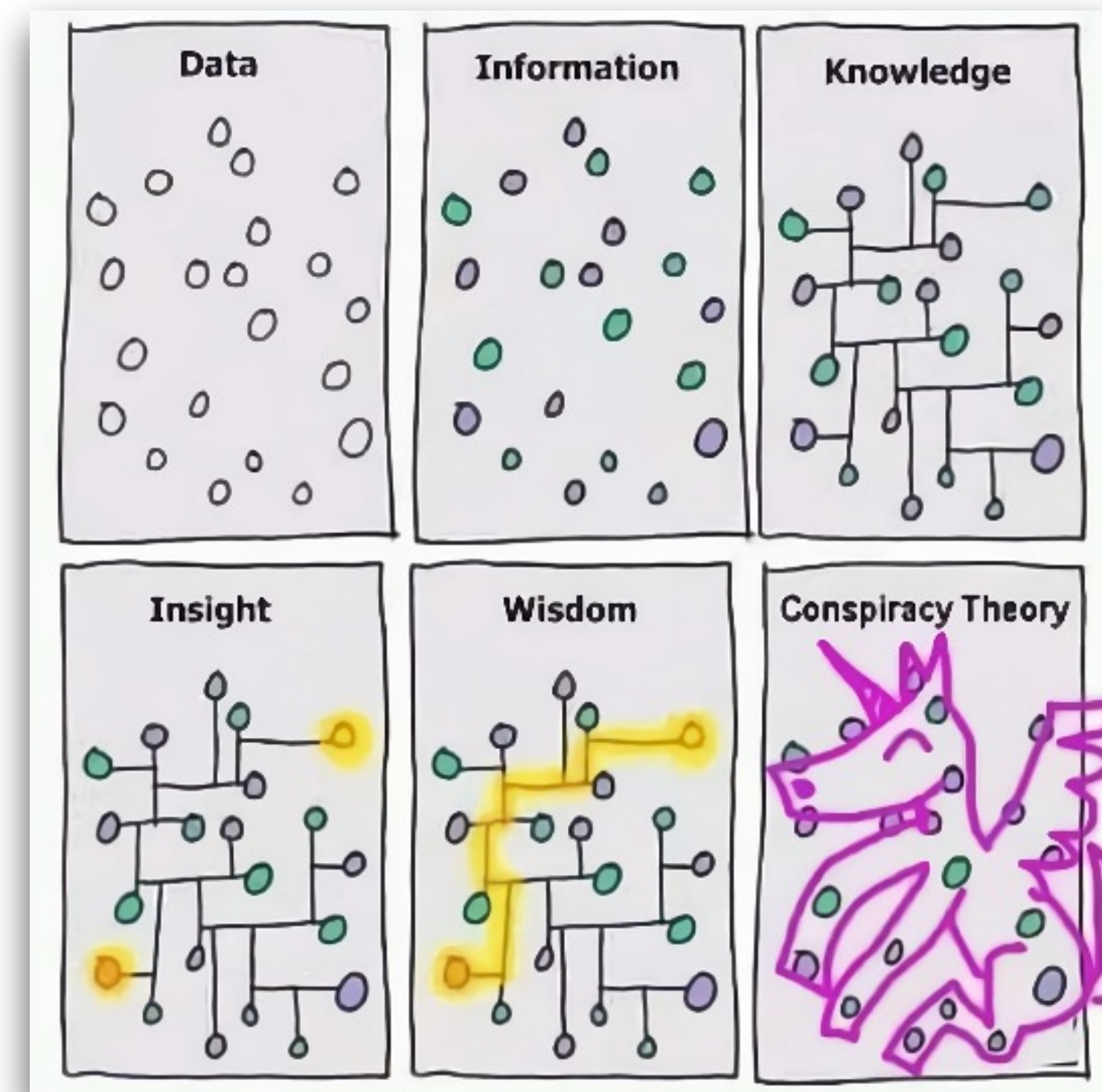
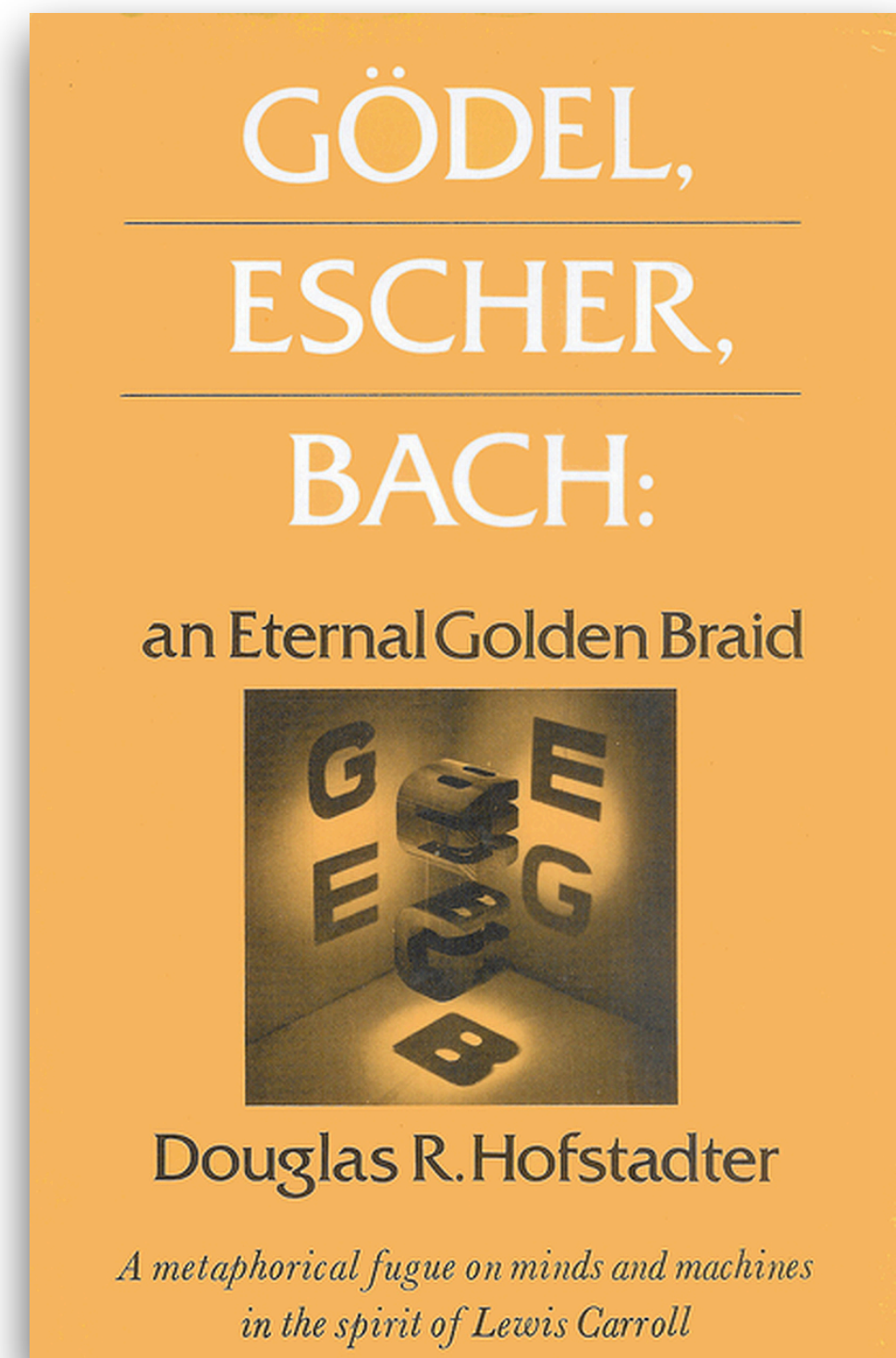
- Goto

$$\frac{\Gamma, x:\tau \rightarrow \tau' \vdash E : \tau}{\Gamma \vdash \text{catch } x E : \tau}$$

$$\frac{\Gamma, A \rightarrow B \vdash A}{\Gamma \vdash A} \text{Pierce} = \frac{\Gamma \vdash (A \rightarrow B) \rightarrow A}{\Gamma \vdash A}$$

- Which one is more intuitive? Restricted vs unrestricted control flow?
 - Dijkstra, “Letters to the Editor: Goto Statement Considered Harmful”, CACM, 1968

Connect the Dots



Summary

- Two sides of one coin: program & proof
- An eye-opening framework leading to theoretical and practical advances in CS & logic
- Theory
 - Program $\leftrightarrow$ proof, type $\leftrightarrow$ proposition
 - Intuitionistic logic $\leftrightarrow$ functional computation, classical logic $\leftrightarrow$ control
- Practice
 - Interactive theorem prover (proof assistant)
 - Program synthesis & proof synthesis